Public Transport Tracker

Architecture Design Document (HLD + LLD)

Full Stack Microservice — React + Spring Boot

Option A: Resilience & Offline Mode

Senior Developer Promotion Submission

1. Executive Summary

This document presents the complete High-Level Design (HLD) and Low-Level Design (LLD) for the Public Transport Tracker — a production-ready, full-stack microservice application that provides real-time transit information to end users via a React-based Single Page Application backed by Spring Boot microservices.

Selected Architecture Option: Option A — Resilience & Offline Mode

- ► In-memory cache with configurable TTL per data type
- ► Stale-data serving strategy when upstream transit APIs are unavailable
- ► Explicit degradation: UI shows cache age and offline indicators

Why NOT Option B (Observability): Observability is a cross-cutting concern and is included as a non-functional add-on (Spring Actuator, structured logging) without being the primary focus.

Why NOT Option C (Data Reasoning): ETA/route planning algorithms are included in the Route Planning service, but resilience is the harder production-readiness challenge for a transit app that depends on frequently unreliable third-party APIs.

2. Problem Statement Analysis

2.1 Functional Requirements

Requirement	Implementation Approach
Accept city/route identifier	REST endpoint: GET /api/v1/transit?city={city}&route={route}
Real-time vehicle locations	Transit Data Service polls MTA/Transit.land, normalizes GTFS-RT feed
Arrival predictions (ETA)	Route Planning Service: Dijkstra-based ETA + schedule offset calculation
Service disruptions & alerts	Alerts Service: threshold rules engine; Push via SSE to React UI
Route planning & alternatives	Route Planning Service: alternative route graph traversal
Crowding levels	Alerts Service: crowding field from GTFS vehicle positions feed
Graceful API failure handling	TTL cache layer + stale-data fallback + degraded-mode flag in response
Offline mode with toggle	React UI toggle → API returns cached data; SW caches last-good response

2.2 Conditional Alert Rules

Condition	Alert Message
delay > 15 minutes	"Significant delays - Plan accordingly"
service disruption flag = true	"Service alert - Check alternative routes"
crowding_level = HIGH	"Vehicle at capacity - Consider next service"
weather_impact = true	"Weather impact on schedule"

2.3 Non-Functional Requirements

- Performance: P95 API latency < 500ms under normal load; cache hit serves in < 50ms
- Availability: Graceful degradation — never returns 5xx to client; always returns data (live or cached)
- Security: API keys encrypted at rest; HTTPS only; rate limiting at gateway
- Portability: Dockerized, port 8080, 12-Factor App compliant
- Testability: TDD (unit tests) + BDD (scenario tests) with ≥ 80% coverage target
- Observability: Structured JSON logs, Spring Actuator health/readiness endpoints

3. High-Level Design (HLD)

3.1 System Architecture Overview

The application follows a layered microservice architecture with clear separation of concerns across five layers:

Layer	Components	Responsibility
Presentation	React SPA (Browser)	User interface, route search, map display, alert rendering, offline toggle
API Gateway	Spring Cloud Gateway	Request routing, authentication, rate limiting, CORS
Business Services	Transit Data, Route Planning, Alerts, Health services	Core domain logic, data aggregation, alert evaluation
Cache / Infra	ConcurrentHashMap TTL Cache, Secret Manager	Resilience, stale data, API key management
External Integration	MTA API, Transit.land, TfL API	Raw transit data sources (unreliable, rate-limited)

3.2 Architecture Decisions (ADRs)

ADR-01: No Database Required

Decision: Use in-memory ConcurrentHashMap as the sole data store. Transit data has a short natural TTL (30–300 seconds). A database would add operational complexity without benefit. Cache survives for the life of the JVM process — a restart causes a brief cold-start where all requests go directly to upstream APIs.

ADR-02: Microservice Decomposition

Four domain-aligned services are deployed within a single Spring Boot application (modular monolith approach for simplicity, but designed for eventual extraction). Each service has its own package, interface contract, and test suite. This avoids the operational overhead of multiple deployment units while preserving separation of concerns.

ADR-03: Option A — Resilience First

Transit APIs (MTA, TfL) are third-party services with documented rate limits and occasional outages. A real-time transit app that returns 503 during API downtime is unusable. Serving 2-minute-stale data is far preferable to returning an error. The TTL cache with stale-while-revalidate is therefore the primary non-functional investment.

ADR-04: Server-Sent Events for Alert Push

Alerts (delays, disruptions) are time-sensitive and must reach the user without polling. SSE (Server-Sent Events) provides a lightweight one-way push channel from the Spring Boot server to the React client, avoiding the complexity of WebSockets while still being real-time.

3.3 Component Diagram (Text Representation)

System Component Map

```
[Browser]
  │ HTTPS
  ▼
[React SPA] ── REST/SSE ──► [API Gateway :8080]
                    │
        ┌───────────┼───────────┐
        ▼           ▼           ▼
 [Transit Data Service]  [Route Planning Service]  [Alerts Service]
        │           │           │
        └───────────┼───────────┘
                    │
            [In-Memory TTL Cache]
                    │
        ┌───────────┼───────────┐
        ▼           ▼           ▼
   [MTA API]   [Transit.land API]   [TfL API]
 (api.mta.info)  (transit.land/api/v2)  (api.tfl.gov.uk)
```

4. Sequence Diagrams

4.1 Happy Path — Route Search

Sequence: GET /api/v1/transit?city=NYC&route=A

```
User        React UI      API Gateway    Transit Service   MTA API   Cache
 |           |            |              |                |         |
 | Search route |          |              |                |         |
 |──────────────────────►|              |                |         |
 |           | GET /transit |              |                |         |
 |           |──────────────────────►|                |         |
 |           |            | routeRequest() |                |         |
 |           |            |──────────────────────►|         |
 |           |            |              | cache.get()    |         |
 |           |            |              |──────────────────────────────►|
 |           |            |              |  MISS          |         |
 |           |            |              |◄──────────────────────────────|
 |           |            |              | fetchVehicles()|         |
 |           |            |              |──────────────────────►|         |
 |           |            |              | GTFS-RT data   |         |
 |           |            |              |◄──────────────────────|         |
 |           |            |              | cache.put()    |         |
 |           |            |              |──────────────────────────────►|
 |           |            |  TransitDTO    |                |         |
 |           |            |◄──────────────────────|         |         |
 |           | 200 + JSON   |              |                |         |
 |           |◄──────────────────────|                |         |
 |  Renders map |          |              |                |         |
 |◄──────────────────────|                |                |         |
```

4.2 Degraded Path — API Unavailable (Cache Fallback)

Sequence: Upstream API failure — stale data returned

```
User        React UI      API Gateway    Transit Service   MTA API   Cache
 |           |            |              |                |         |
 | Search route |          |              |                |         |
 |──────────────────────►|              |                |         |
 |           | GET /transit |              |                |         |
 |           |──────────────────────►|                |         |
```

```
|          |          |──────────────────►|          |       |
|          |          |          | fetchVehicles()|       |
|          |          |          |──────────────────►|       |
|          |          |          |  TIMEOUT/5xx  |       |
|          |          |          |◄──────────────────|       |
|          |          |          | cache.getStale()|       |
|          |          |          |──────────────────────────────►|
|          |          |          |  stale entry  |       |
|          |          |          |◄──────────────────────────────|
|          |          |  staleDTO +   |          |       |
|          |          |  dataAge: 120s |          |       |
|          |          |◄──────────────────|          |       |
|          |  200 (stale)  |          |          |       |
|          |◄──────────────────|          |          |       |
| Shows banner:  |          |          |          |       |
| 'Data 2m old'  |          |          |          |       |
|◄──────────────────|          |          |          |       |
```

5. Low-Level Design (LLD)

5.1 Package Structure

Spring Boot Project Layout

```
com.transport.tracker
├── api
│   ├── TransitController.java        — REST endpoints
│   ├── AlertsController.java         — SSE endpoint
│   └── dto
│       ├── TransitResponseDTO.java
│       ├── VehicleLocationDTO.java
│       ├── RouteDTO.java
│       └── AlertDTO.java
├── service
│   ├── TransitDataService.java       — Interface
│   ├── impl
│   │   ├── TransitDataServiceImpl.java
│   │   ├── RouteplanningServiceImpl.java
│   │   └── AlertsServiceImpl.java
│   └── cache
│       └── TTLCacheService.java      — ConcurrentHashMap + TTL
├── client
│   ├── MtaApiClient.java
│   ├── TransitLandClient.java
│   └── TflApiClient.java
├── config
│   ├── AppConfig.java
│   ├── SecurityConfig.java
│   └── OpenApiConfig.java
└── exception
    ├── ApiUnavailableException.java
    └── GlobalExceptionHandler.java
```

5.2 Key Class Responsibilities

Class / Interface	Responsibility
TransitController	Handles GET /api/v1/transit and GET /api/v1/routes. Delegates to service layer. Returns TransitResponseDTO with dataAge field.

TransitDataServiceImpl	Orchestrates data fetch: check TTL cache → if miss, call API client → normalize response → put in cache → return DTO.
TTLCacheService<K,V>	Generic in-memory cache. ConcurrentHashMap<K, CacheEntry<V>>. CacheEntry holds value + expiresAt. getStale() returns value even if expired. Background thread evicts fully expired entries.
AlertsServiceImpl	Evaluates alert rules: delay threshold, disruption flag, crowding level, weather impact. Returns List<AlertDTO> sorted by severity.
RoutePlanningServiceImpl	Builds a graph from GTFS stop sequence data. Runs Dijkstra to find shortest travel time route and 1–2 alternatives.
MtaApiClient	Wraps MTA REST API. Decrypts API key from env. Timeout: 3s. Throws ApiUnavailableException on failure.
GlobalExceptionHandler	@ControllerAdvice. Catches ApiUnavailableException and triggers stale data path instead of returning 5xx.

5.3 TTL Cache Design

The cache is the core resilience mechanism. Key design decisions:

- Key: String composed of city + routeId (e.g. "NYC:A")
- TTL: Vehicle locations — 30s; ETA predictions — 60s; Alerts — 120s; Route data — 300s
- Eviction: Lazy eviction on get() + background ScheduledExecutorService every 60s
- Stale reads: getStale(key) returns value regardless of TTL — used during API outage
- Thread safety: ConcurrentHashMap ensures no data races under concurrent requests
- dataAge: response includes seconds since cache entry was last refreshed — shown in UI

5.4 React Frontend Component Tree

React Component Hierarchy

```
App
├── AppRouter
│   ├── HomePage
│   │   ├── SearchBar (city + route input)
│   │   └── OfflineToggle (local state → API header)
│   ├── DashboardPage
│   │   ├── LiveMap (Leaflet map + vehicle markers)
│   │   ├── AlertsPanel (SSE subscription → alert cards)
│   │   ├── ETATable (arrival predictions)
│   │   └── StaleBanner (shown when dataAge > threshold)
│   └── RoutePlannerPage
```

```
│       ├── RouteForm
│       └── RouteResultsList
├── hooks
│   ├── useTransitData(city, route)   — SWR/fetch + cache
│   ├── useAlertStream()              — SSE EventSource
│   └── useOfflineMode()              — localStorage flag
└── services
    └── transitApi.js                 — Axios instance + interceptors
```

5.5 API Contract (OpenAPI Summary)

Endpoint	Method	Description
GET /api/v1/transit	GET	Fetch real-time vehicle locations, ETAs, alerts for a city/route. Params: city, route, offline (bool)
GET /api/v1/routes/plan	GET	Route planning with alternatives. Params: origin, destination, city
GET /api/v1/alerts/stream	GET (SSE)	Server-Sent Events stream of real-time alerts for a route
GET /actuator/health	GET	Spring Boot health check (liveness)
GET /actuator/readiness	GET	Readiness probe — confirms upstream connections available
GET /api/v1/cache/status	GET	Returns current cache entries, TTL status, hit/miss ratio

6. Resilience & Degradation Strategy (Option A Detail)

6.1 Degradation Levels

Level	Trigger	Behaviour
LIVE	All upstream APIs healthy, cache fresh	Real-time data served. dataAge shown but no banner.
STALE	Upstream API timeout or 5xx, cache has entry	Stale data served. UI shows 'Data may be X minutes old' banner. Cache age shown.
DEGRADED	Cache miss + API unavailable	Empty vehicle list returned. Alert: 'Live data unavailable'. Static route info served if available.
OFFLINE (user toggle)	User manually enables offline mode	All requests served from cache (or last-known). No API calls made. Clear offline indicator in UI.

6.2 Cache TTL Policy

Data Type	TTL Strategy
Vehicle positions	30 seconds live TTL; serve stale up to 5 minutes
ETA predictions	60 seconds live TTL; serve stale up to 10 minutes
Service alerts	120 seconds live TTL; serve stale up to 30 minutes
Route/stop data	300 seconds live TTL; serve stale indefinitely (static-ish)

7. CI/CD & Infrastructure

7.1 CI/CD Pipeline Stages

GitHub Actions / Jenkins Pipeline

```
Stage 1: Code Checkout
  └── git clone + branch validation

Stage 2: Build
  ├── Backend:  mvn clean package -DskipTests
  └── Frontend: npm install && npm run build

Stage 3: Test
  ├── Backend:  mvn test (JUnit + integration tests)
  └── Frontend: npm run test (Jest + React Testing Library)

Stage 4: Code Quality
  └── SonarQube / Checkstyle scan

Stage 5: Docker Build
  └── docker build -t transport-tracker:${BUILD_NUMBER} .

Stage 6: Push to Registry
  └── docker push registry/transport-tracker:${BUILD_NUMBER}

Stage 7: Deploy
  ├── Dev:     auto-deploy on every push
  ├── Staging: auto-deploy on merge to main
  └── Prod:    manual approval gate
```

7.2 Dockerfile Summary

Multi-Stage Docker Build

```
Stage 1 (build): Maven image → compile JAR
Stage 2 (run):   OpenJDK 17 slim → copy JAR only

EXPOSE 8080
ENV JAVA_OPTS=-Xms256m -Xmx512m
```

```
HEALTHCHECK CMD curl -f http://localhost:8080/actuator/health


Image size target: < 200MB
Non-root user: appuser (UID 1001)
```

7.3 Multi-Environment Config (12-Factor)

Config Key	Dev	Prod
SPRING_PROFILES_ACTIVE	dev	prod
MTA_API_KEY	dev-key (mock)	encrypted secret from vault
CACHE_TTL_VEHICLE_SECONDS	10	30
LOG_LEVEL	DEBUG	INFO (JSON structured)
API_RATE_LIMIT_RPS	100	1000

8. Security Design

- API keys stored as environment variables only — never hardcoded
- Keys encrypted at rest using AES-256 via application-level SecretManager utility
- HTTPS enforced at gateway — HTTP redirects to HTTPS
- CORS whitelist: only known frontend origin domains
- Rate limiting at API Gateway: 100 req/s per IP (configurable)
- No sensitive data in logs — API keys masked via custom log filter
- Spring Security for endpoint protection — public: /api/v1/transit; protected: /actuator/*

9. Design Patterns & SOLID Principles

Pattern / Principle	Where Applied
Single Responsibility (SRP)	Each service class has one job: TransitDataService only fetches/normalizes. AlertsService only evaluates rules.
Open/Closed (OCP)	TransitDataService is an interface. New API providers (e.g. BART) implement the interface — no existing code changes.
Dependency Inversion (DIP)	Controllers depend on service interfaces, not implementations. Implementations injected via Spring @Autowired.
Strategy Pattern	TTL eviction policy is pluggable. CacheEvictionStrategy interface allows LRU, TTL, or size-based strategies.
Decorator Pattern	CachedTransitDataService wraps the real TransitDataService, adding cache-check logic transparently.
Factory Pattern	TransitClientFactory creates the correct API client (MTA, TfL, Transit.land) based on city identifier.
Circuit Breaker (Conceptual)	ApiUnavailableException triggers immediate fallback to cache — prevents cascade failures from slow APIs.
HATEOAS	API responses include hypermedia links: _links.self, _links.alternatives, _links.alerts for discoverability.

10. Testing Strategy

10.1 Backend (JUnit + Mockito)

- Unit tests: each service class tested in isolation; API clients mocked with Mockito
- Integration tests: Spring Boot test slice (@WebMvcTest) for controller layer
- Cache tests: verify TTL expiry, stale reads, concurrent access
- BDD: Cucumber scenarios for alert rule engine (Given delay=20m, When evaluated, Then expect 'Significant delays')

10.2 Frontend (Jest + React Testing Library)

- Component tests: StaleBanner renders when dataAge > threshold
- Hook tests: useTransitData returns cached data when API fails
- E2E: Playwright or Cypress for critical user journeys (search → view map → see alert)

10.3 Coverage Targets

Layer	Target Coverage
Service layer (business logic)	≥ 85%
Cache layer (resilience paths)	≥ 90%
Alert rules engine	100% (all rules covered)
React components	≥ 70%
API contract (contract tests)	All endpoints

Appendix: Technology Stack Summary

Layer	Technology Choice
Frontend Framework	React 18 + TypeScript + Vite
UI Components	Tailwind CSS + Radix UI primitives
Map Library	Leaflet.js (open source, no key required)
State Management	React Query (SWR) for server state; Zustand for UI state
Backend Framework	Spring Boot 3.x + Java 17
API Documentation	Springdoc OpenAPI (Swagger UI at /swagger-ui.html)
Build Tool	Maven (backend) + Vite (frontend)
Testing (Backend)	JUnit 5 + Mockito + Cucumber (BDD)
Testing (Frontend)	Jest + React Testing Library + Playwright
CI/CD	GitHub Actions primary + Jenkins job config included
Containerization	Docker (multi-stage build) — port 8080
Logging	Logback + Logstash encoder (structured JSON logs)
Health / Metrics	Spring Boot Actuator (/health, /readiness, /metrics)